

MC613 – Relatório 3

Projeto: Controlador DRAM

Rodrigo Banin Ferraz de Camargo (238257)

Junho de 2026

Sumário

1	Planejamento	2
1.1	Diagramas da etapa inicial	3
2	Estrutura do Código	6
2.1	Arquivos sintetizáveis	6
2.2	Arquivos de simulação	6
2.3	Descrição dos componentes	6
3	Implementação	6
3.1	Decisões centrais de implementação	6
3.2	Trechos essenciais de código	7
3.2.1	Buffer de comando no <code>reader</code>	7
3.2.2	Arbitragem no estado <code>READY</code> do controlador	7
3.2.3	Captura de leitura na borda de descida	8
3.2.4	Leitura automática pós-escrita no <code>dram_iface</code>	8
3.2.5	Direção do barramento e integração no top-level	8
4	Validação (Testes)	9
4.1	Teste de componente: <code>tb_dram_iface.vhd</code>	9
4.2	Teste de componente: <code>tb_dram_controller.vhd</code>	10
4.3	Teste de integração: <code>tb_dram_top_level.vhd</code>	11
5	Análise de Execução	12
5.1	Captura do dado de leitura (o problema central)	12
5.2	Inicialização longa e parametrização por <i>generic</i>	12
5.3	Barramento <code>DRAM_DQ</code> bidirecional	13
5.4	PLL: simulação versus síntese	13
5.5	Sincronização de entradas e handshake	13

1 Planejamento

O planejamento foi orientado pelos requisitos da entrega final: implementar um controlador SDRAM funcional, integrá-lo com a interface da placa DE1-SoC e validar por simulação os fluxos obrigatórios (INIT, READ, WRITE, REFRESH e WRITE seguido de READ automático).

A arquitetura foi dividida em camadas para reduzir a complexidade e permitir a validação incremental de cada parte:

- **dram_iface**: camada de interação com o usuário (SW/KEY/HEX), que traduz ações em requisições de leitura/escrita por meio do handshake `req/ready`.
- **reader**: pequeno registrador (buffer) que segura uma requisição estável entre a interface e a FSM de protocolo até o controlador consumi-la.
- **dram_controller**: camada de protocolo SDRAM, responsável pelos comandos (ACTIVATE, READ, WRITE, PRECHARGE, AUTO REFRESH, LOAD MODE) e por toda a temporização crítica.
- **dram_top_level**: integração com a PLL, com a interface de usuário e com os pinos físicos da SDRAM.

Essa separação permitiu uma evolução incremental: primeiro validar a emissão de requisições da interface, depois validar a temporização do controlador, e por fim validar a integração completa entre os dois.

1.1 Diagramas da etapa inicial

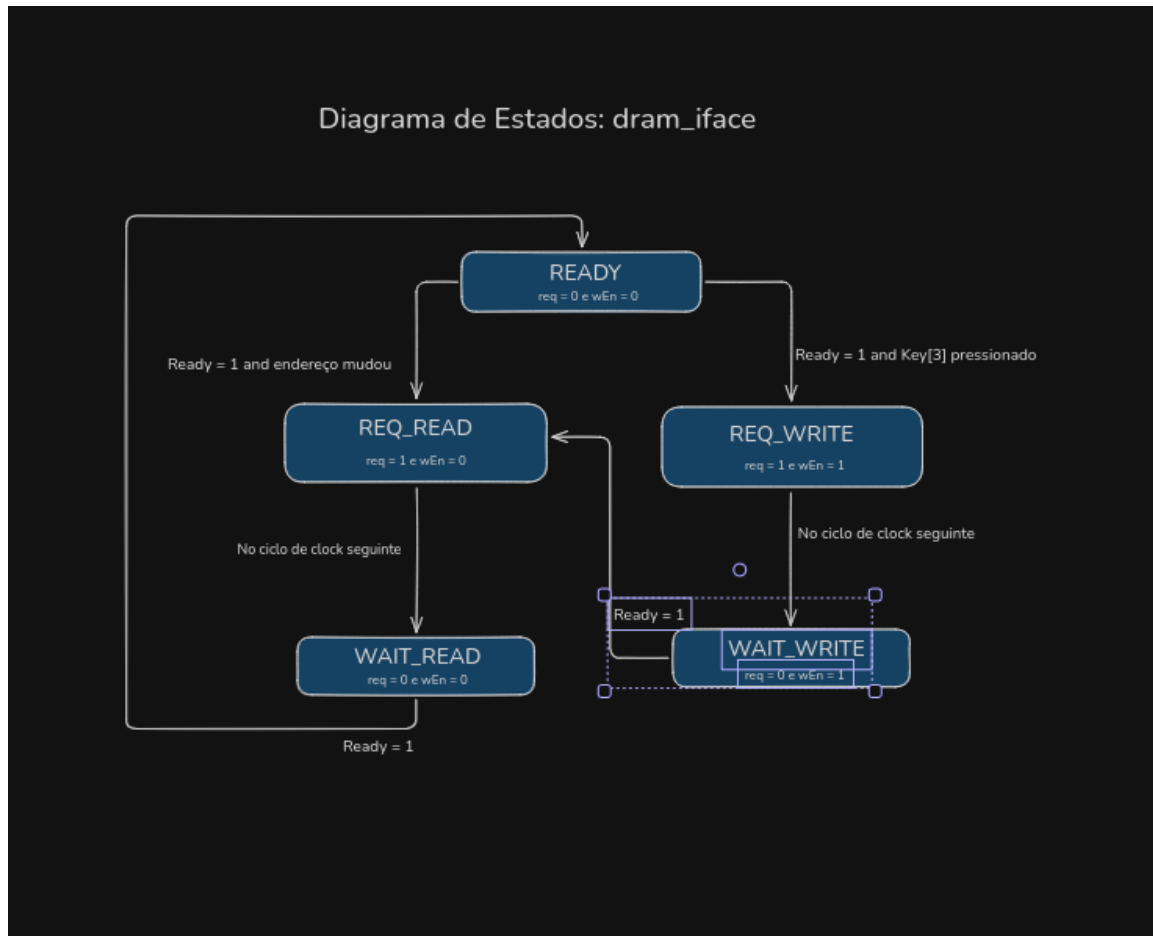


Figura 1: Diagrama de estados do módulo `dram_iface`.

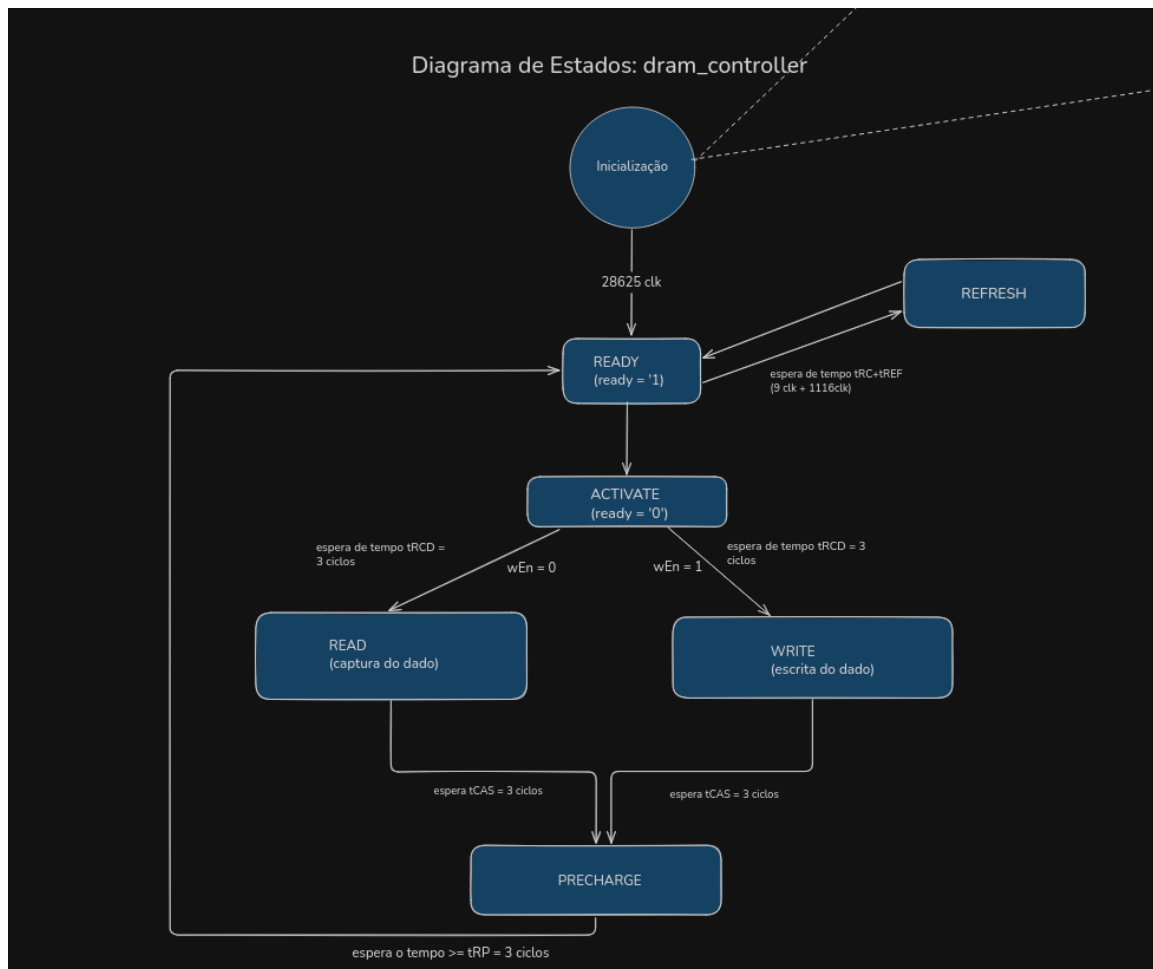


Figura 2: Diagrama de estados do módulo `dram_controller`.

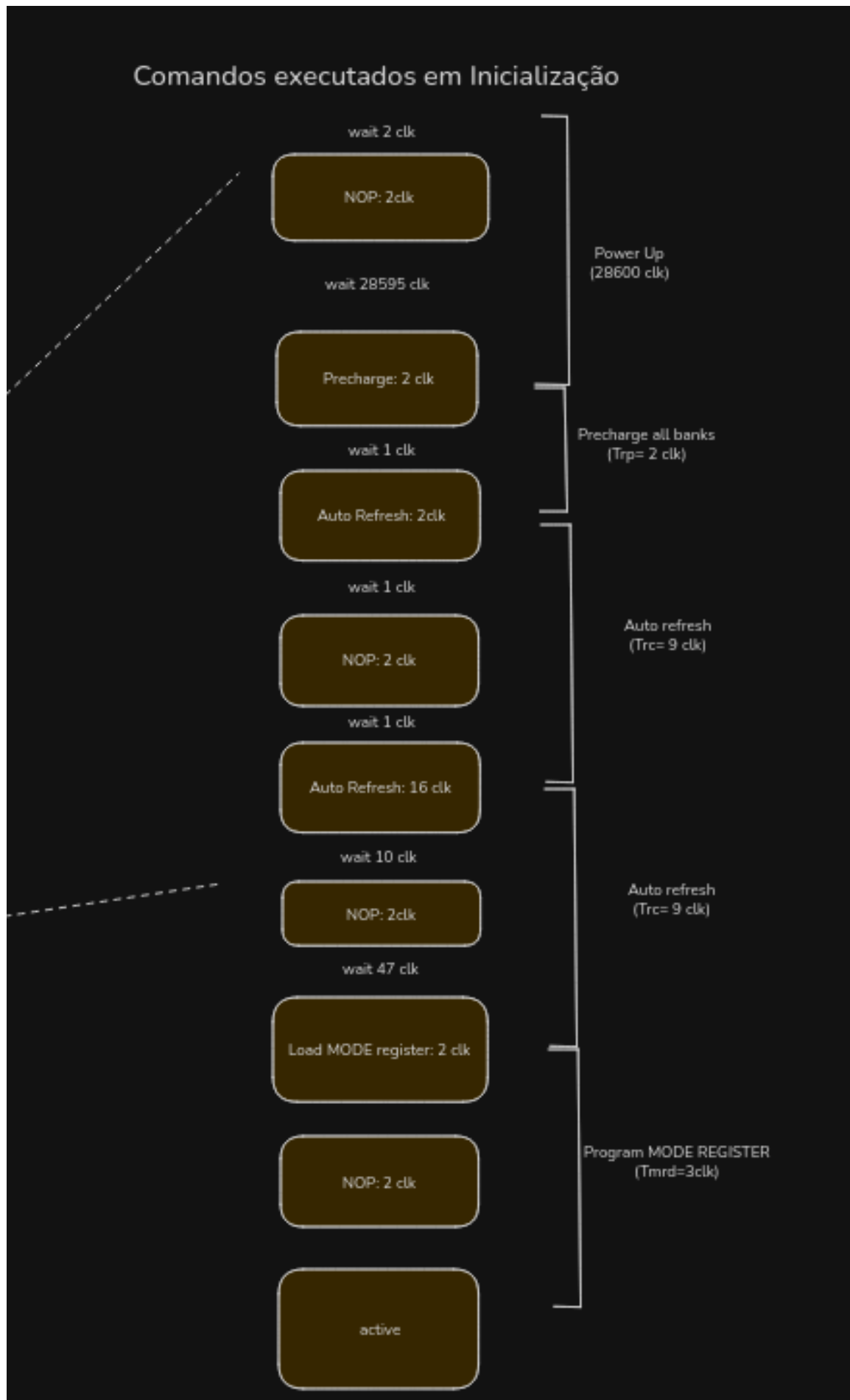


Figura 3: Sequência de inicialização da SDRAM (PRECHARGE ALL, 8 AUTO RE-FRESH, LOAD MODE REGISTER).

2 Estrutura do Código

2.1 Arquivos sintetizáveis

- `dram_iface.vhd`
- `reader.vhd`
- `dram_controller.vhd`
- `dram_top_level.vhd`
- `pll.vhd` / `pll.qip` (megafunção de PLL da Altera, gera o clock interno a partir de `CLOCK_50`).

2.2 Arquivos de simulação

- `pll_sim_stub.vhd`: substituto da PLL **apenas em simulação** (o ghdl não elabora a megafunção Altera). Mantém a mesma entidade `pll` e gera um clock de aproximadamente 143 MHz, levantando `locked` após alguns ciclos. Nunca é sintetizado.
- `tb_dram_iface.vhd`: valida isoladamente a FSM de interface.
- `tb_dram_controller.vhd`: valida isoladamente o protocolo e a temporização do controlador.
- `tb_dram_top_level.vhd`: valida a integração ponta a ponta do sistema completo.

2.3 Descrição dos componentes

dram_iface: gera `req/wEn`, detecta escrita por borda de descida de `KEY[3]`, dispara leitura ao detectar mudança de endereço e faz leitura automática após cada escrita. Também sincroniza `SW/KEY` com dois flip-flops (anti-metaestabilidade) e dirige os displays de 7 segmentos (`HEX0`, `HEX1`, `HEX4`, `HEX5`).

reader: captura uma requisição de comando, mantém endereço/dado/modo de escrita estáveis e só libera a FSM principal quando o controlador confirma o consumo por `cmd_ack`.

dram_controller: consome os comandos bufferizados pelo **reader** e executa o protocolo SDRAM (inicialização, `ACTIVATE`, `READ`, `WRITE`, `PRECHARGE`, `AUTO REFRESH` e `LOAD MODE`), aplicando todas as esperas de temporização e capturando o dado de leitura do barramento `DRAM_DQ`. A espera de power-up foi parametrizada por um *generic* `INIT_WAIT_CYCLES` (padrão 28600).

dram_top_level: integra a PLL, a interface de usuário e os pinos físicos da SDRAM, roteando o vetor de comando de 26 bits do controlador para os pinos `DRAM_*` e tratando o barramento bidirecional `DRAM_DQ`.

3 Implementação

3.1 Decisões centrais de implementação

- FSM com estados intermediários de espera dedicados para cada tempo crítico (`tRCD`, `tRP`, `CAS latency`, `tRFC`, `tMRD`);

- uso do **reader** como registrador de comando entre a interface e a FSM de SDRAM, com handshake por `cmd_ack`;
- codificação explícita dos comandos SDRAM por constantes de 26 bits (comando + endereço no mesmo vetor `address`);
- **captura do dado de leitura na borda de descida do clock** — a correção mais importante do projeto (Seção 5);
- parametrização da espera de inicialização por *generic*, para acelerar a simulação sem alterar o comportamento sintetizado;
- controle explícito da direção do barramento DRAM_DQ: o top-level só o dirige no ciclo exato do comando WRITE.

3.2 Trechos essenciais de código

3.2.1 Buffer de comando no reader

Listing 1: Captura do comando ate o acknowledge do controlador (reader.vhd).

```

1  -- Se o controlador reconhece o comando, limpa o pendente
2  if cmd_ack = '1' then
3      pending_reg <= '0';
4  end if;
5
6  -- Captura nova requisicao (inclusive no mesmo ciclo do ack, p/
   fluxo continuo)
7  if req_in = '1' and (pending_reg = '0' or cmd_ack = '1') then
8      pending_reg <= '1';
9      wEn_reg      <= wEn_in;
10     addr_reg      <= addr_in;
11     data_reg      <= data_in;
12 end if;
```

3.2.2 Arbitragem no estado READY do controlador

Listing 2: Refresh tem prioridade sobre uma nova requisicao externa (dram_controller.vhd).

```

1  when st_ready =>
2      ready <= '1';
3      if refresh_due = '1' then
4          ready      <= '0';
5          address     <= CMD_PREALL;
6          timer_cycles <= TRP_WAIT_CYCLES; timer_start <= '1';
7          op_state    <= st_refresh_trp;
8      elsif buffered_req = '1' then
9          ready      <= '0';
10         cmd_ack     <= '1';
11         active_addr <= buffered_addr;
12         active_wen  <= buffered_wen;
```

```

13     active_data   <= buffered_data;
14     address       <= command_with_address(CMD_ACTIVATE,
15                                     buffered_addr,
16                                     row_address(buffered_addr
17                                     ));
16     timer_cycles  <= TRCD_WAIT_CYCLES; timer_start <= '1';
17     op_state      <= st_activate_wait;
18 end if;

```

3.2.3 Captura de leitura na borda de descida

Listing 3: A janela valida do dado cai entre as bordas de subida; capturar na descida acerta (dram_controller.vhd).

```

1  -- DRAM_CLK = not pll_clk, entao a borda de descida de clk coincide
   com a
2  -- borda de subida do clock do chip. A 143 MHz, o dado valido (tAC
   apos a
3  -- borda) cai ENTRE as subidas do sistema; capturar na descida le o
   valor
4  -- correto em vez de amostrar o barramento ainda em alta impedancia
   (0xFF).
5  dq_capture : process(clk, rst)
6  begin
7      if rst = '1' then
8          dq_in_reg <= (others => '0');
9      elsif falling_edge(clk) then
10         dq_in_reg <= data_in;
11     end if;
12 end process;

```

3.2.4 Leitura automática pós-escrita no dram_iface

Listing 4: Ao fim da escrita, a propria interface dispara um READ de confirmacao (dram_iface.vhd).

```

1  when WAIT_WRITE_ST =>
2      if ready = '1' then
3          req_reg <= '1';    -- dispara leitura de confirmacao
4          wen_reg <= '0';
5          state   <= REQ_READ_ST;
6      end if;

```

3.2.5 Direção do barramento e integração no top-level

Listing 5: O top-level so dirige DRAM_DQ no comando WRITE e instancia o controlador (dram_top_level.vhd).

```

1  -- So dirige DRAM_DQ durante o WRITE (CS=0, RAS=1, CAS=0, WE=0);
   senao alta-Z

```



```

2 DRAM_DQ <= (x"00" & controller_write_data)
3     when (dram_cmd(18)='0' and dram_cmd(17)='1' and
4           dram_cmd(16)='0' and dram_cmd(15)='0')
5     else (others => 'Z');
6
7 controller_i : dram_controller
8     generic map (INIT_WAIT_CYCLES => INIT_WAIT_CYCLES)
9     port map (
10        clk => pll_clk, rst => rst, SW => SW, KEY => KEY,
11        data_in => DRAM_DQ(7 downto 0), write_data_in =>
12            iface_write_data,
13        data_out => controller_data_out,
14        adress => dram_cmd, addr_in => iface_address,
15        write_data_out => controller_write_data,
16        req => req_sig, wEn => wen_sig, ready => ready_sig);

```

4 Validação (Testes)

A validação foi organizada em três níveis, do mais isolado ao mais integrado, cada um com seu testbench autoverificável (todos terminam imprimindo todos os cenários passaram ou falham com mensagem clara):

1. `tb_dram_iface`: a FSM de interface, sozinha;
2. `tb_dram_controller`: o protocolo e a temporização do controlador, com um modelo de SDRAM fiel ao tempo do chip;
3. `tb_dram_top_level`: o sistema completo, estimulado como na placa.

4.1 Teste de componente: `tb_dram_iface.vhd`

Objetivo: validar a FSM de interface isoladamente, sem controlador SDRAM nem memória reais. O foco é a *sequência de operações* que a interface emite a partir de switches e botões.

Metodologia: um *controlador falso* (`fake_ctrl`) responde ao handshake — a cada `req='1'` ele registra a operação (`wEn`, `address`, `write_data`) em um log, baixa `ready` por alguns ciclos e devolve um `read_data` fixo (0x5A). Verificar a sequência registrada (e não ciclos contados) torna o teste imune aos dois estágios de sincronização de `SW/KEY` e à latência variável do handshake. A função `addr_of` reproduz o mapeamento de `SW` para o endereço de 26 bits, permitindo prever o endereço esperado.

Cenários cobertos:

- `op0` — WRITE 0x00: o reset auto-limpa o endereço selecionado (`wEn=1`, dado 0x00);
- `op1` — READ automático de read-back da limpeza (`wEn=0`);
- `op2` — READ disparado pela mudança dos switches de endereço (`wEn=0`, endereço `addr_of`);
- `op3` — WRITE 0x0A ao pressionar `KEY[3]` (dado `=SW[3:0]`);

- op4 — READ automático de read-back após a escrita (wEn=0).

Um *watchdog* de 50 μ s falha o teste caso a sequência não se complete.

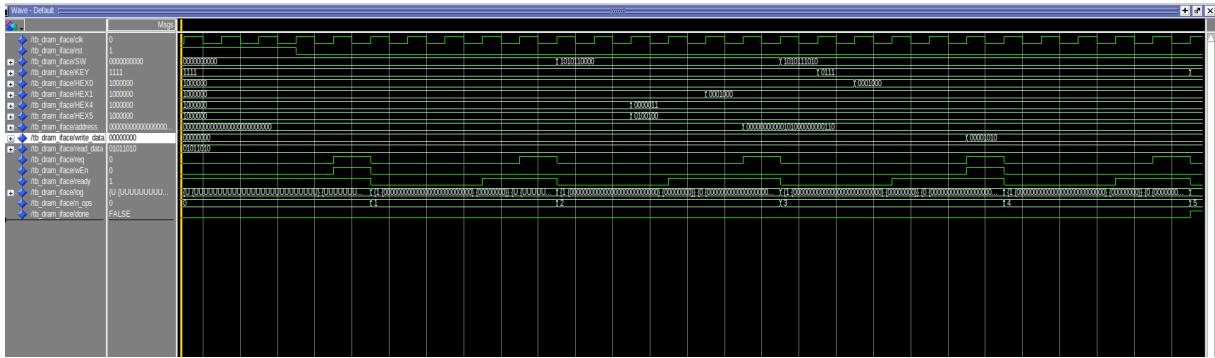


Figura 4: Simulação de `tb_dram_iface`: os cinco pulsos de `req` com os `wEn/address/write_data` esperados.

Por que isto prova a correção: a forma de onda mostra exatamente os cinco pulsos de `req` na ordem esperada, com `wEn=1` apenas nas escritas (`op0`, `op3`) e o endereço/dado corretos em cada uma. Isso comprova que a lógica de decisão da interface — limpeza no reset, leitura ao trocar de endereço, escrita por `KEY[3]` e read-back automático — está correta, independentemente do controlador real. Console: `tb_dram_iface: todos os cenários passaram`.

4.2 Teste de componente: `tb_dram_controller.vhd`

Objetivo: validar o protocolo SDRAM e, sobretudo, a temporização de captura de leitura, sem a interface de usuário.

Metodologia — fidelidade ao hardware: o testbench modela o SDRAM real nos pinos `DRAM_*` com período de 7 ns (≈ 143 MHz), `DRAM_CLK = not clk`, **CAS latency 3** e **tAC = 5400 ps** (o dado de leitura só fica válido tAC após a borda do chip, modelado com `DRAM_DQ <= tb_dq_data after TAC`). Sem essa fidelidade o teste passaria com um valor de captura errado, mascarando o bug que só aparece na placa. A espera de inicialização é encurtada apenas na simulação via `generic map (INIT_WAIT_CYCLES => 50)`, de modo que toda a atividade fica visível desde o início (o teste todo cabe em cerca de 9 μ s). O testbench decodifica os comandos (`dram_cmd_dec`), conta-os (`n_activate`, `n_read`, ...) e tem guardas de contenção de barramento e *watchdog*.

Cenários cobertos:

- INIT: inicialização completa (PRECHARGE, 8 AUTO REFRESH, LOAD MODE) e subida de `ready`;
- WRITE 0x5A no endereço A e READ de retorno (deve ler 0x5A);
- independência de endereços: WRITE 0x3C em B, READ B (0x3C) e READ A novamente (ainda 0x5A);
- AUTO REFRESH espontâneo durante o período ocioso.

O procedimento `dram_op` respeita a regra de handshake (esperar `ready='0'` e depois `ready='1'`), evitando o erro de atraso por um.

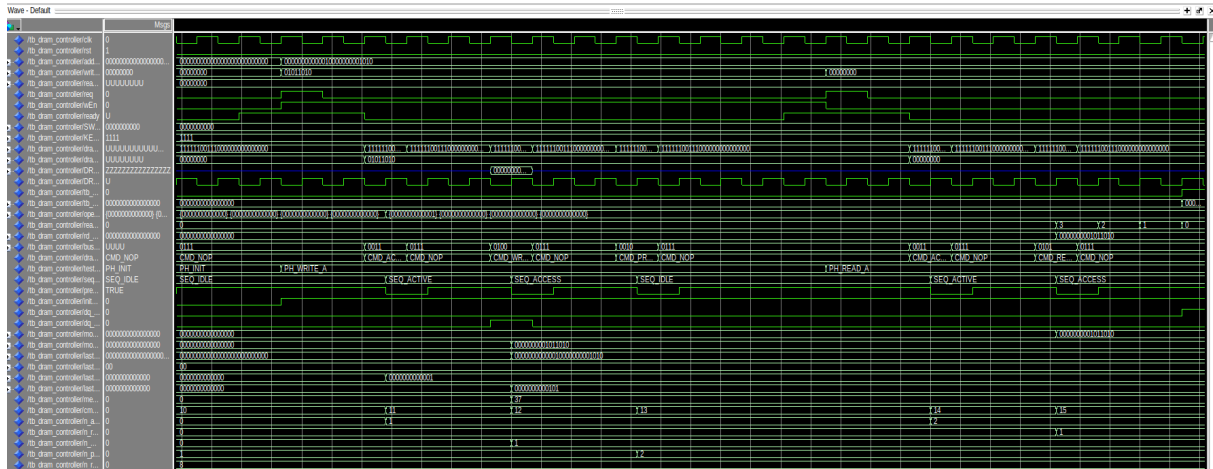


Figura 5: Simulação de `tb_dram_controller`: sequência ACTIVATE → READ/WRITE → PRECHARGE, o barramento DRAM_DQ e a captura do dado dentro da janela válida.

Por que isto prova a correção: a forma de onda evidencia, para cada operação, a sequência de comandos correta e o instante em que o dado válido aparece em DRAM_DQ (tAC após a borda do chip) sendo amostrado na borda de descida. Os READs devolvem exatamente os valores escritos (0x5A/0x3C) e o READ A repetido continua 0x5A, provando armazenamento correto e independência de endereços; o AUTO REFRESH aparece sozinho no ocioso. Console: `tb_dram_controller: todos os cenários passaram.`

4.3 Teste de integração: `tb_dram_top_level.vhd`

Objetivo: validar o sistema completo (`iface` → `reader` → `controller` → SDRAM → `display`) estimulando apenas `CLOCK_50`, `SW` e `KEY`, como um usuário na placa, e conferindo o valor mostrado em `HEX1`.

Metodologia: a PLL de simulação gera ≈ 143 MHz (não repassa os 50 MHz, para não mascarar a temporização de leitura) e levanta `locked` após alguns ciclos. O testbench decodifica os comandos a partir dos pinos reais `DRAM*_N`, sincroniza as verificações com eventos observáveis (`wait_cmd` espera `CMD_LOAD_MODE`, `CMD_WRITE`, `CMD_READ`, `CMD_AUTO_REFRESH`) e compara `HEX1` com o padrão de 7 segmentos esperado. A inicialização é encurtada via `generic map` (`INIT_WAIT_CYCLES => 50`) para deixar a forma de onda legível desde $t=0$. O testbench é compatível com VHDL-2002, rodando tanto no ghdl quanto na RTL Simulation do Quartus (ModelSim-Altera).

Cenários cobertos (a “história”):

- RESET e INIT: solta o reset e espera o LOAD MODE REGISTER (init concluído), confirmando os 8 AUTO REFRESH;
- escreve 7 no endereço 1 via `KEY[3]` → `HEX1` mostra 7;
- seleciona o endereço 2 (nunca escrito) → `HEX1` mostra 0;
- volta ao endereço 1 → `HEX1` continua 7 (persistência);

- AUTO REFRESH espontâneo durante o ocioso.

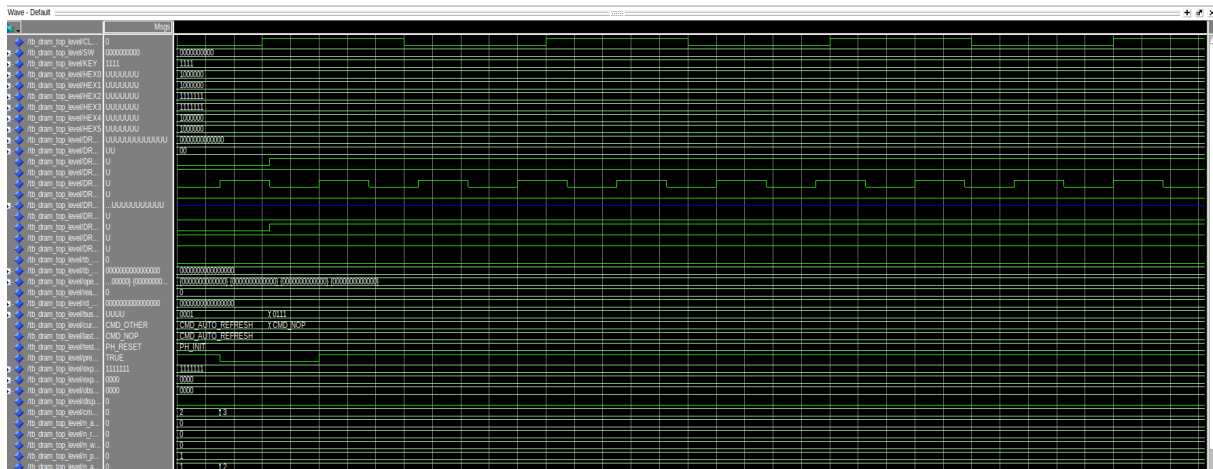


Figura 6: Simulação de `tb_dram_top_level`: reset, init, escrita via `KEY[3]`, leituras e o valor final em HEX1.

Por que isto prova a correção: como nada interno é injetado, a forma de onda demonstra o caminho completo funcionando: após o init, escrever 7 no endereço 1 faz HEX1 exibir 7; mudar para um endereço virgem mostra 0; e **voltar ao endereço 1 reexibe 7**, provando que o dado foi de fato gravado na SDRAM e relido (persistência). O AUTO REFRESH espontâneo confirma a manutenção da memória. Console: `tb_dram_top_level: todos os cenarios passaram.`

5 Análise de Execução

5.1 Captura do dado de leitura (o problema central)

A maior dificuldade foi a leitura retornar 0xFF em vez do dado gravado. A causa era de temporização: a 143 MHz, o dado válido chega tAC após a borda do clock do chip e essa janela cai *entre* as bordas de subida do clock do sistema — amostrar na subida lia o barramento ainda em alta impedância. A solução foi capturar DRAM_DQ na **borda de descida** de clk (que, com DRAM_CLK = not pll_clk, coincide com a subida do chip), usar READ_CAPTURE_CYCLES = 5 e o *mode register* 0x230 (CAS latency 3, burst length 1). Com CAS latency 2 (0x220) ou captura na subida, o teste lê 0xFF — exatamente o sintoma original.

5.2 Inicialização longa e parametrização por *generic*

A espera obrigatória de power-up do SDRAM dura 28600 ciclos (cerca de $200\mu s$ a 143 MHz), em que o barramento fica legitimamente em NOP. Em simulação isso deixava a forma de onda “parada” (apenas os clocks oscilando) até $200\mu s$. Expor essa espera como o *generic* INIT_WAIT_CYCLES (padrão 28600 na síntese, sobrescrito para 50 nos testbenches) resolveu o problema sem alterar o comportamento sintetizado: a atividade passou a ser visível desde $t=0$.

5.3 Barramento DRAM_DQ bidirecional

O barramento de dados é bidirecional, com risco de contenção. A solução foi o top-level dirigir `DRAM_DQ` *apenas* no ciclo do comando `WRITE` (deixando-o em alta impedância no restante), enquanto o modelo de SDRAM nos testbenches só dirige durante a leitura (após `tAC`). Os testbenches incluem guardas concorrentes que falham se houver contenção (`X` no barramento ou dois *drivers* simultâneos).

5.4 PLL: simulação versus síntese

O ghdl não elabora a megafunção de PLL da Altera. Foi criado o `pll_sim_stub.vhd`, que mantém a mesma entidade e gera um clock de ≈ 143 MHz com `locked` atrasado — garantindo que a simulação exerça a mesma frequência e o mesmo caminho de reset por `locked` do hardware real. Na síntese e na RTL Simulation do Quartus usa-se a PLL real; o stub é exclusivo da simulação em ghdl.

5.5 Sincronização de entradas e handshake

`SW` e `KEY` são assíncronos: amostrá-los direto a 143 MHz pode levar a FSM à metaestabilidade. A interface os sincroniza com dois flip-flops. No handshake `req/ready`, foi essencial esperar `ready='0'` (operação aceita) antes de `ready='1'` (operação concluída); checar apenas `ready='1'` levaria a ler o dado da operação anterior (erro de atraso por um).